

RV-MalTrace

What should the group believe after this update?

Scope: controlled Genesys2/CVA6 tracing and semantic reconstruction. Not a malware detector or production transport claim.

artifact root: results/evaluation/genesys2-cva6/current

**TRACE
EVENTS
FIRST**

problem

evidence

decision

The report has to answer six questions

Every slide should help answer one of them.

Q1

Research problem: low-level trace must support high-level behavior claims

Q2

Why it matters: software references are visible and must not be the only evidence path

Q3

Core conclusion: controlled reconstruction works; detector accuracy is not claimed

Q4

Evidence: trace tests, source labels, attribution fixtures, board windows, repro checks

Q5

Boundary: no real-malware accuracy, production transport, or cycle-overhead claim

Q6

Decision needed: what claim is worth submitting, and which gap to close next?

What is the research problem?

— Hardware trace is low-level; malware behavior claims are high-level.

SOFTWARE OBSERVER

Rich semantics

Useful reference data, but visible to the workload.

OUT-OF-BAND TRACE

Hardware path

Commit-time trace starts below user-mode observer checks.

Why is this worth studying?

— Software logs are useful references, but they are part of the measured system.

OBSERVATION

Software traces are visible

They are excellent references, but adversarial workloads can observe or react to them.

INTERPRETATION

Hardware events are not behavior

A PC, trap, or syscall transition still needs process and executable context.

EVIDENCE

Sources can be confused

Reference logs must validate results without being reported as hardware output.

What is the current core conclusion?

RV-MalTrace supports controlled behavior reconstruction, not detector accuracy.

Current conclusion

For controlled RISC-V Linux workloads, RV-MalTrace can reconstruct behavior summaries from CVA6 trace records while labeling supporting evidence sources.

SUPPORTED

trace ordering, syscall pairing, source labels, code attribution, reproducibility

NOT CLAIMED

real-malware accuracy, payload equivalence, production throughput, cycle overhead

Which evidence supports the conclusion?

— The evidence is useful only if each claim keeps its source and scope.

TRACE

Directed tests support event ordering and syscall entry/return pairing

SOURCE

Reconstruction summaries mark hardware, ELF, runtime-map, and reference fields

ATTRIB.

Local fixtures cover PIE/ASLR, dynamic objects, fork/exec, stripped binaries

BOARD

122 accepted CVA6 windows support scoped empirical claims

REPRO

Manifest and SHA-256 checks make the current evidence package reviewable

How does the evidence flow?

Trace records, code maps, runtime maps, and references stay separate.



SEPARATE

hardware trace is kept separate from reference labels

TAG

code and behavior claims carry source labels

Evidence 1: trace correctness

— The trace contract is tested before semantic claims are made.

CONTROL	retire, branch, jump
SYSCALL	entry and qualified return
CONTEXT	trap, CSR, SATP, privilege
ACCOUNT	ARG_MEM, marker, drop records

Evidence 2: source-labeled reconstruction

— Derived fields are marked as hardware, ELF, runtime map, or reference.

hardware trace

event sequence, traps, syscall transitions

ELF metadata

binary identity, sections, symbols

runtime map

process scope and loaded objects

reference log

expected behavior for comparison

The point is not more labels; it is preventing reference evidence from being reported as hardware recovery.

Evidence 3: code attribution

— ELF identity, symbols, syscall sites, and runtime maps constrain attribution.

CODE MAP

ELF identity

hash, load ranges, sections,
symbols

JOIN

Trace to code

PC evidence plus
process/runtime scope

LIMITATION

No source-line claim

DWARF/source-line
attribution needs board
evidence

Evidence 4: board runs

— 122 accepted CVA6 windows support scoped empirical claims.

122

accepted board runs

12

samples

126

attempts

4

failed retained

0

drops / wraps /
gaps

Evidence 5: controlled workloads

— The workload set checks behavior reconstruction, not real-world coverage.

safe baseline



4 samples

42 accepted reps

surrogate



8 samples

80 accepted reps

benign controls



5 samples

scoped controls

Evidence 6: behavior checks

— Perfect scoped metrics validate reconstruction, not detector accuracy.

1.0

syscall recall

1.0

precision

1.0

argument recon.

0.0

benign FPR

Controlled workload analysis only. Not detection accuracy.

What are the boundaries?

— Unsupported topics stay as limitations or future work.

WRITE

hardware-assisted behavior tracing and analysis

WRITE

35T end-to-end prototype; scoped CVA6 evidence

LIMIT

synthetic and safety-controlled malware-derived behaviors

DO NOT

real-malware accuracy, payload equivalence, production detector

What else could explain the result?

— The current evidence rules out some failures, but not all external validity concerns.

ALT 1

Reference logs may supply too much semantic information; source labels expose that boundary

ALT 2

Local code-attribution fixtures may not equal board-native source-line evidence

ALT 3

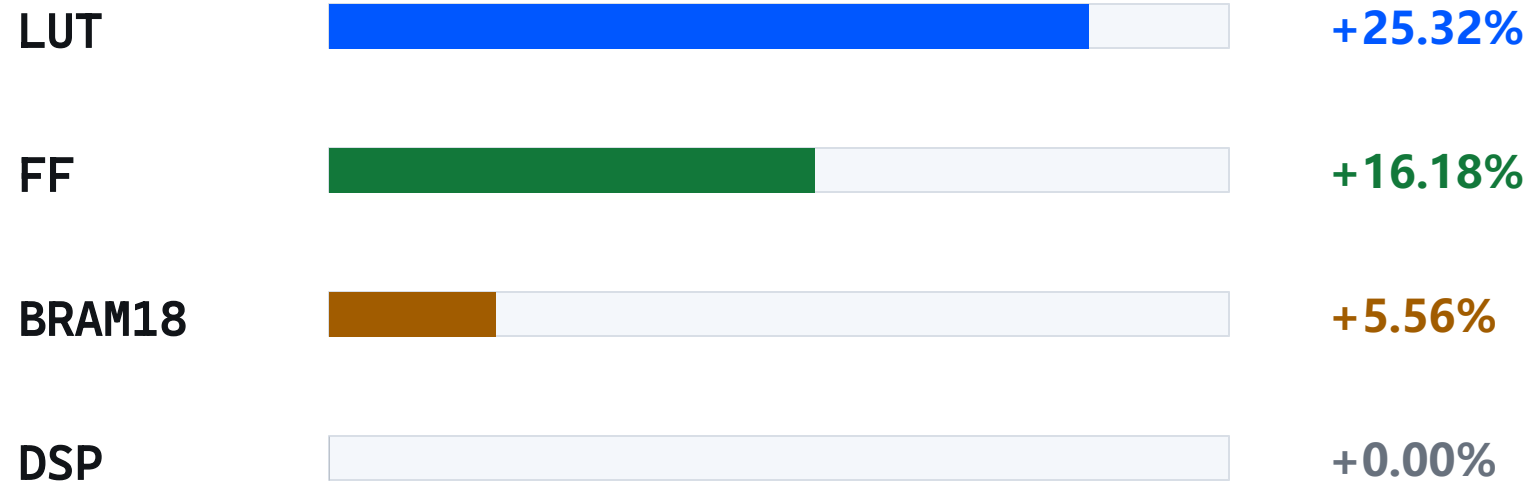
Controlled workloads may not predict in-the-wild malware behavior coverage

ALT 4

Resource deltas do not establish runtime slowdown without cycle-overhead measurement

What cost is measured?

— Resource and timing evidence support implementation cost, not runtime slowdown.



Timing MET, slack 0.177 ns. Cycle-level runtime overhead remains unsupported.

What should the group help decide?

— The next work is not more running; it is claim selection and one evidence gap.

DECIDE	Is the submission claim trace-backed behavior reconstruction, not malware detection?
DECIDE	Which single evidence gap matters most: board-native source lines, overhead, or transport?
REMOVE	Move real-malware accuracy and payload-equivalence claims to limitations/future work
NEXT	Convert remaining prose claims into final figures and claim-evidence tables

The artifact package is reviewable

— The paper should cite reproducible evidence, not untracked experiment memory.

MANIFEST

current artifact root accepted

ARCHIVE

3160 files, 316.9 MB, SHA d5b806...433b

CHECKERS

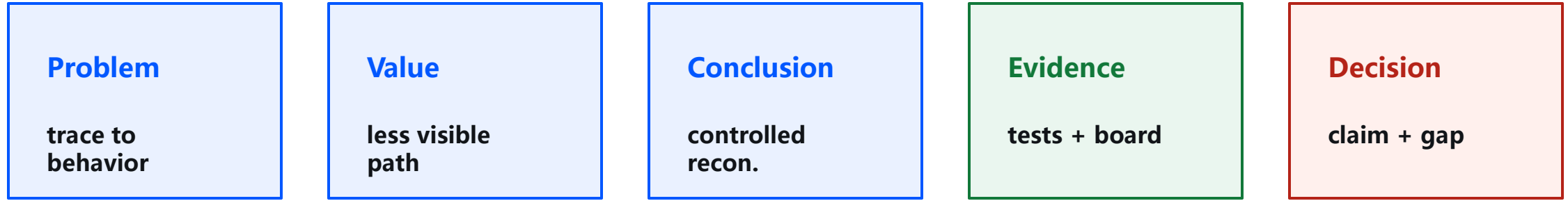
repro:quick and documented full gates pass

PAPER

NDSS draft builds; figures and quantitative tables still need final polish

Paper narrative

— Problem -> conclusion -> evidence -> boundary -> decision.



Streaming/DMA remains future work

Readiness and target profiles are not throughput evidence.

TARGET	p99 = 0.0215755 event bytes/cycle
REQUIRED	0.0323633 event bytes/cycle, about 1.618 MB/s at 50 MHz
CURRENT	sustained streaming = 0; noninterference = false

Workload roster

— Sample names are appendix material, not main-story material.

SAFE

hello_write; file_open_read_write; fork_exec; illegal_instruction

SURROGATE

file_scan; batch_open_read_write; self_copy_sim; abnormal_syscall_sequence

SURROGATE

illegal_trap; process_chain; dynamic_executable_memory; anti_debug_like

BENIGN

hello; cat; clock_status; getdents_only; mmap_rw

Resource detail

Resource/timing claim is supported; runtime overhead is not.

	LUT	FF	BRAM18	DSP
baseline	84,923	56,491	108	27
trace	106,428	65,634	114	27
delta	+25.32%	+16.18%	+5.56%	+0